SOFTWARE TEST PLAN (STP)

for

Snake Game

Prepared by:
Charan (PES1UG25CS856), Vrunda (PES1UG24CS541)
Ved (PES1UG24CS522), Tushar (PES1UG24CS504)

Document Control

Revision History

Version	Date	Author(s)	Change Summary	Approval
1.0	12-10-2026	Charan, Vrunda, Ved, Tushar	Initial draft aligned to SRS v1.0	Pending

Approvals

Role	Name	Signature / Email	Date
Project guide / Instructor	Adithya Balasubramanyam		
Team representative	Vrunda		

1. Introduction

1.1 Purpose

This Software Test Plan (STP) describes how the Snake Game will be tested. It defines what will be tested, how, in which environment, by whom and when, and how results will be traced back to the requirements in the Snake Game Software Requirements Specification (SRS) v1.0. It is the reference for the test team, the developers and the course evaluators.

1.2 Scope

The plan covers verification of the single-player desktop Snake Game (C++ with SFML): starting and restarting a round, keyboard control and movement, food and score handling, collision and game-over behaviour, safe window close, and the non-functional and security requirements in the SRS (FR-01 to FR-18, NFR-01 to NFR-06, SR-01 to SR-05). Testing covers the unit, integration, system and acceptance levels described in Section 5.

Out of scope are the items listed in Section 4.

1.3 References

- Snake Game – Software Requirements Specification (SRS), Version 1.0 (Draft for review), 12 October 2026.
- IEEE Std 829-2008, IEEE Standard for Software and System Test Documentation (structure of this plan).
- IEEE SRS template for the SE Mini-Project (document layout and numbering style).
- Software Engineering Mini-Project deliverable instructions provided by the course.
- SFML documentation for the version selected by the team (see Section 6.2).

1.4 Definitions, Acronyms and Abbreviations

Term	Meaning
STP	Software Test Plan
SRS	Software Requirements Specification
FR / NFR / SR / SO	Functional Requirement / Non-Functional Requirement / Security Requirement / Security Objective
TC	Test Case (TC-01 to TC-14, as referenced in the SRS RTM)
RTM	Requirements Traceability Matrix
SFML	Simple and Fast Multimedia Library
FPS	Frames per second
Round	One play session from the start action until a fatal collision
Fatal collision	Snake head hits the play-area boundary or the snake's own body
ASan / UBSan	AddressSanitizer / UndefinedBehaviorSanitizer (compiler runtime checkers)
N / P / F / NA	Test status: Not Run / Pass / Fail / Not Applicable

2. Test Items

The following items are placed under test. Version identifiers will be fixed when the first test build is tagged in the repository.

Test item	Description	Version / identifier
Snake Game executable	Compiled game built from the repository using the documented compiler and SFML version.	[TBD – git tag / commit hash]
Game logic modules	Game control, snake movement, food handling, scoring and collision detection (SRS NFR-04 modular split).	[TBD]

Test item	Description	Version / identifier
Input handler	Keyboard and window event handling, including direction validation.	[TBD]
Renderer / UI	Play area, snake, food, score and game-over display.	[TBD]
README / build documentation	Required tools, SFML dependency, build/run steps and control instructions (NFR-02, NFR-06).	[TBD]

3. Features to be Tested

Requirement IDs	Feature	Verified by	Test case
FR-01 to FR-03	Start a round; bounded play area; snake placed fully inside	System / integration	TC-01
FR-04 to FR-06	Keyboard direction input; snake movement; 180° reversal rejection	Unit, integration, system	TC-02 (FR-06 also TC-13)
FR-07 to FR-09	Food placement and detection; snake growth	Unit, integration, system	TC-03
FR-10, FR-11	Score update; replacement food	Unit, integration, system	TC-04
FR-12 to FR-15	Boundary and self-collision; stop on fatal collision; game-over message and final score	Unit, integration, system	TC-05
FR-16, FR-17	Restart after game over; reset of snake, direction and score	System	TC-06
FR-18, SR-05	Safe application close and shutdown on window close	System	TC-07
NFR-01	Performance: average ≥ 30 FPS over a 60 s run	System measurement	TC-08
NFR-02	Usability: controls and restart documented	Acceptance (reviewer)	TC-09
NFR-03	Reliability: 10 consecutive start/restart cycles without crash	System	TC-10
NFR-04	Maintainability: responsibilities in identifiable modules	Code review	TC-11
NFR-05, NFR-06	Compatibility (clean build and launch); build documentation in README	Clean build, README walkthrough	TC-12
SR-01, SR-04 (SO-01)	Unsupported-key handling; validation of direction changes	System, unit, sanitizer	TC-13
SR-02, SR-03 (SO-02)	No OS command execution; no unrelated file access	Code review, tool trace	TC-14

4. Features Not to be Tested

The following are excluded from this test effort, with the reason:

Feature / area	Reason for exclusion
Pause/resume, sound effects, high-score persistence	Not specified in the SRS and therefore not required. If the team adds them, the SRS, this plan and the RTM will be revised together.
Multiplayer, online accounts, networking, database	Not part of the product (SRS §1.2 and §2.1: online services and database functionality are out of scope).
Gamepad, mouse and touch input	Keyboard is the only specified control method (SRS §2.5 and §3.2).
Operating systems / compilers other than the documented environment	NFR-05 requires build and launch only in the documented environment.

Feature / area	Reason for exclusion
Internals of SFML and the OS	Third-party components are assumed correct; only their use by the game is tested.
Formal penetration testing and cryptography	The game is a local, offline application with no data exchange; security validation is limited to Section 5.1.
Accessibility (colour-blind modes, screen readers)	No such requirement exists in the SRS.

5. Test Approach / Strategy

Testing combines automated unit tests on the game logic with manual and scripted tests on the running game. Test cases TC-01 to TC-14 are the formal test cases referenced by the SRS RTM; each lists its own steps, data and expected result (Section 11). The approach for security is given first, in Section 5.1, followed by the four test levels.

5.1 Security Validation

Objective. Confirm that the game meets the security objectives of the SRS: SO-01 (unsupported keyboard input and invalid game-state transitions do not crash the application) and SO-02 (the application stays limited to local-game behaviour, with no unintended access to unrelated files or execution of operating-system commands).

Threat view. The game has no network, accounts or stored user data, so the realistic attack surface is (a) keyboard and window events supplied by the player, (b) files the process opens, and (c) any code path that could launch an external command. Validation is therefore focused on input robustness, safe execution and file access, and clean shutdown.

SR	Security requirement (summary)	Validation technique	Tools	Test case
SR-01	Ignore unsupported keyboard keys without crashing	Press every undocumented key, then 10 s of random key mashing; try invalid state changes (start key mid-game, direction key at game-over). Run with a sanitizer build.	Manual, ASan/UBSan	TC-13
SR-02	No OS command execution in response to player input	Code review and text search for system(), popen(), exec*, fork, ShellExecute; trace input paths to confirm none reaches them.	grep, cppcheck, review checklist	TC-14
SR-03	No reading/modifying unrelated user files	Review all file-open calls; monitor file activity through a full round; allow only game assets and required libraries.	strace / Process Monitor, review	TC-14
SR-04	Validate direction changes against current game state	Test direct and double-key reversals; unit-test the direction-validation function with all 16 direction pairs.	Unit tests, manual	TC-13 (also TC-02)
SR-05	Handle window-close events and shutdown safely	Close during play, at game over, with a key held and right after launch; confirm no unhandled crash or sanitizer report.	Manual, ASan/UBSan	TC-07

Security pass criteria. Every SR verification step passes, no crash occurs in any security run, and the sanitizer-enabled build reports no memory or undefined-behaviour errors. Any security failure is logged as a Critical defect (Section 5.5).

5.2 Test Levels

5.2.1 Unit Testing

Performed by the developers on individual classes or functions of the game logic, without a window: direction validation (including all reversal pairs), snake step and growth, food placement avoiding the snake, score update, and wall and self-collision checks. Framework: a C++ unit-test framework such as Catch2 or GoogleTest (to be confirmed by the team). Random food placement is made testable through a seedable random generator or an injected position. Unit tests support TC-02 to TC-05 and TC-13.

5.2.2 Integration Testing

Verifies that the modules work together through the game loop, following the SAD: input → direction validation → calculate the next head position → check collision → apply the valid move (including food pickup, growth and score update) → render. Integration is checked incrementally (input + snake, snake + food/score, snake + collision + game state, game controller + renderer) using the debug build and logging of game-state transitions (playing → game over → playing). A collision found in the "check collision" step must stop the move from being applied. Supports TC-01 to TC-06.

5.2.3 System Testing

Performed on the complete built game on the agreed test computer, black-box through the keyboard and window. Covers the full functional flow, safe shutdown (TC-07), performance (TC-08), reliability over repeated cycles (TC-10), build and compatibility (TC-12) and the security checks (TC-13, TC-14).

5.2.4 Acceptance Testing

Final check against the acceptance criteria written in the SRS (Sections 4–6). Includes a usability session where a reviewer outside the team uses only the README and in-game instructions (TC-09), and a final walk-through of TC-01 to TC-14 with the course instructor or team representative.

5.3 Item Pass / Fail Criteria

- A test case passes only when every step's actual result matches the expected result.
- The release is accepted when all High-priority test cases pass, no Critical or Major defect is open, and every SRS requirement has at least one passed test case in the RTM.
- Statuses stay N (Not Run) until the test is actually executed; no result is recorded as P without evidence in the test log.

5.4 Suspension and Resumption Criteria

- **Suspend** testing when the build does not launch, a crash blocks more than half of the planned test cases, or the environment (SFML/compiler) changes mid-cycle.
- **Resume** after a fixed build is delivered and a quick smoke test (TC-01 plus a short TC-02 run) passes. Affected tests are then re-run.

5.5 Defect Management

Defects are logged as GitHub Issues in the project repository using labels for severity, linked to the failing test case and requirement ID. Fixes are verified by re-running the failed test and the related regression tests.

Severity	Meaning	Example
Critical	Crash, hang or data/security failure; blocks testing	Game crashes on window close; command execution via input
Major	Required behaviour is wrong; no workaround	Snake can reverse into itself; score does not increase
Minor	Behaviour is wrong but a workaround exists	Food occasionally appears at the edge cell
Trivial	Cosmetic or documentation issue	Typo in the README; label misaligned

6. Test Environment

6.1 Hardware

Item	Requirement
Test computer	Desktop or laptop with a graphical display and keyboard. Record exact CPU, RAM, GPU and OS in the test log: [TBD – agreed test machine, also used for NFR-01].
Display	Resolution large enough for the full game window: [TBD – confirm window size].
Input device	Standard keyboard.

6.2 Software

Item	Requirement
Operating system	[TBD – Windows 10/11 or Ubuntu 22.04/24.04 LTS; use the same one as documented in the README]
Compiler	[TBD – e.g. g++ 11+ / MSVC 2022 / clang], C++17 or as documented
SFML version	[TBD – e.g. SFML 2.6.x or 3.x; must match the README]
Build system	[TBD – CMake / Makefile]
Version control	Git and GitHub (organisation repository, project board, team members as collaborators)

6.3 Tools and Test Data

Tool / data	Use
Unit-test framework (Catch2 or GoogleTest)	Automated unit tests of game logic
AddressSanitizer / UBSan (-fsanitize=address,undefined)	Detect memory and undefined-behaviour errors during TC-07, TC-10 and TC-13
cppcheck, grep	Static checks and code search for SR-02 and SR-03
strace (Linux) / Process Monitor (Windows)	Observe file access for SR-03 (TC-14)
FPS counter or log output	Measure frame rate for NFR-01
GitHub Issues / Projects	Defect tracking and sprint backlog
Spreadsheet (test_cases.xlsx)	Execution log with status, date, tester and evidence
Configuration values (to be frozen in the README)	Control keys, start/restart action, window/grid size, initial snake length and position, food value, move speed: [TBD – confirm]

7. Test Schedule and Deliverables

7.1 Schedule

Sprints are optional at this stage. After the checkpoint, activities are scheduled per sprint; exact dates are added when a sprint is started.

#	Activity	Owner	Target	Status
1	Baseline SRS v1.0 and approve this Test Plan; first deliverables ready for checkpoint	All members	13-10-2026	In progress
2	Freeze control keys, config values, compiler and SFML version in the README	Developers	Per sprint plan	Pending
3	Write unit tests and prepare test data	Developers	Per sprint plan	Pending
4	Integration testing (incremental, per module)	Developers (integration: Tushar)	Per sprint plan	Pending
5	System testing: TC-01 to TC-12	Tushar / Vrunda	Per sprint plan	Pending
6	Security validation: TC-13, TC-14 and TC-07 (Section 5.1)	All members (cross-review)	Per sprint plan	Pending
7	Defect fixing and regression testing	Developers	Per sprint plan	Pending
8	Acceptance test and usability session (TC-09)	All + external reviewer	Per sprint plan	Pending

#	Activity	Owner	Target	Status
9	Update RTM and issue Test Summary Report	Test Lead	Per sprint plan	Pending

7.2 Test Deliverables

Deliverable	Description	Location
Software Test Plan	This document	docs/Test_Plan_SnakeGame.docx
Test execution log (optional workbook)	Spreadsheet version of the test cases with status, tester and date	docs/test_cases.xlsx [optional]
Unit-test source and results	Automated tests and run output	tests/ (repository)
Test execution log and evidence	Dated results, FPS readings, screenshots, tool output	docs/test_results/ [TBD]
Defect log	GitHub Issues linked to test cases	GitHub repository
Updated RTM	Section 10 statuses updated after execution	This document / SRS Section 8
Test Summary Report	Pass/fail totals, open defects, conclusion	docs/ [TBD]

8. Roles and Responsibilities

Module ownership follows the agreed team allocation. Each member may hold more than one role.

Role	Owner	Responsibilities
Test Lead / Planner	Vrunda	Owns this plan and the RTM; schedules execution; tracks defects; writes the Test Summary Report.
Developer – movement	Charan	Implements snake movement; writes unit tests for movement and direction rules (TC-02); fixes related defects.
Developer – food and scoring	Vrunda	Implements food placement, growth trigger and scoring; writes unit tests (TC-03, TC-04); fixes related defects.
Developer – collisions	Ved	Implements boundary and self-collision detection; writes unit tests including the departing-tail case (TC-05); fixes related defects.
Developer – lifecycle and integration	Tushar	Implements game lifecycle (start, game over, restart, close) and integrates the modules; leads integration and system test execution (TC-01, TC-06, TC-07, TC-08, TC-10); fixes related defects.
Code and security review	All members (cross-review)	Each module is reviewed by a member who did not write it (TC-11, TC-13, TC-14); sanitizer builds and tool traces are run by the reviewer.
Project guide / Instructor	Adithya Balasubramanyam	Reviews and approves the plan; attends acceptance walkthrough.
External usability reviewer	[Classmate / volunteer]	Performs TC-09 using only the README and in-game instructions.

9. Risks and Mitigation

ID	Risk	Likelihood	Impact	Mitigation
R-01	Controls, start/restart action and config values are not fixed, so expected results are ambiguous	High	High	Freeze values in the README before test execution; update TC data fields and review them together.
R-02	Compiler / SFML version differs between team machines, so builds or behaviour differ	Medium	High	Document one version set in the README; verify with a fresh-clone build (TC-12).
R-03	Random food placement makes tests non-repeatable	High	Medium	Use a seedable random generator or debug hook to force food positions.

ID	Risk	Likelihood	Impact	Mitigation
R-04	FPS results vary with machine and background load	Medium	Medium	Use the agreed test machine, close other apps, repeat three runs and record the specification.
R-05	GUI cannot easily be automated, so many tests are manual and slow	High	Medium	Put game logic in testable classes for unit tests; keep manual scripts short and repeatable.
R-06	Limited time before submission; testing squeezed at the end	Medium	High	Start unit tests with development; fix the schedule in Section 7; test each module as it is delivered.
R-07	Team member unavailable	Low	Medium	Cross-train: every role has a backup; all test steps are written down.
R-08	Scope creep (pause, sound, high score) invalidates the plan and RTM	Medium	Medium	Add optional features only by team agreement; update SRS, RTM and tests together.
R-09	Defects found late cause regressions	Medium	Medium	Re-run affected tests plus the smoke subset (TC-01, TC-02) after every fix.

10. Traceability to SRS (RTM)

Every FR, NFR and SR in SRS v1.0 is mapped to at least one test case. Test case IDs match the SRS Section 8 RTM (TC-01 to TC-14). Status: N = Not Run, P = Pass, F = Fail, NA = Not Applicable. Nothing is marked as passed until it has been run.

Requirement ID	Requirement summary	SRS section / module	Test case ID(s)	Status (N/P/F/NA)	Comments
FR-01	Start new round	Section 4 / game control	TC-01	N	
FR-02	Display bounded play area	Section 4 / rendering	TC-01	N	
FR-03	Initial snake placement	Section 4 / snake setup	TC-01	N	
FR-04	Keyboard direction input	Section 4 / input control	TC-02	N	
FR-05	Snake movement	Section 4 / movement	TC-02	N	
FR-06	Reject immediate reversal	Sections 4, 5.1 / input control	TC-02, TC-13	N	
FR-07	Valid food placement	Section 4 / food	TC-03	N	
FR-08	Detect food collection	Section 4 / food	TC-03	N	
FR-09	Grow snake after food	Section 4 / snake	TC-03	N	
FR-10	Update score	Section 4 / score	TC-04	N	
FR-11	Generate replacement food	Section 4 / food	TC-04	N	
FR-12	Boundary collision	Section 4 / collision	TC-05	N	
FR-13	Self-collision	Section 4 / collision	TC-05	N	
FR-14	Stop after fatal collision	Section 4 / game state	TC-05	N	
FR-15	Show game over and final score	Section 4 / user interface	TC-05	N	
FR-16	Restart after game over	Section 4 / game control	TC-06	N	
FR-17	Reset state for new round	Section 4 / game control	TC-06	N	
FR-18	Safe application close	Sections 3, 4 / window events	TC-07	N	

Requirement ID	Requirement summary	SRS section / module	Test case ID(s)	Status (N/P/F/NA)	Comments
NFR-01	Performance at 30 FPS target	Section 5 / rendering loop	TC-08	N	
NFR-02	Controls and restart documented	Section 5 / UI and README	TC-09	N	
NFR-03	10 restart cycles without crash	Section 5 / game lifecycle	TC-10	N	
NFR-04	Responsibilities identifiable in code	Section 5 / code structure	TC-11	N	
NFR-05	Clean build and launch	Section 5 / build environment	TC-12	N	
NFR-06	README has build/run instructions	Section 5 / documentation	TC-12	N	
SR-01	Ignore unsupported keys safely	Section 5.1 / input handling	TC-13	N	
SR-02	No input-driven OS command execution	Section 5.1 / code review	TC-14	N	
SR-03	No unrelated user-file access	Section 5.1 / code review	TC-14	N	
SR-04	Validate direction changes	Section 5.1 / input handling	TC-13	N	
SR-05	Safe window-close and shutdown	Section 5.1 / lifecycle	TC-07	N	

11. Detailed Test Cases

Fourteen detailed test cases cover the functional (TC-01 to TC-07), non-functional (TC-08 to TC-12) and security (TC-07, TC-13, TC-14) requirements, with the same IDs as the SRS Section 8 RTM. Items marked [TBD] depend on decisions the team must freeze in the README. A tabular version with an execution log can be generated from the same data on request.

11.1 Test Case Summary

ID	Title	Requirements	Type	Priority
TC-01	Game start and initial display	FR-01, FR-02, FR-03	Functional	High
TC-02	Keyboard direction input, movement and reversal rejection	FR-04, FR-05, FR-06	Functional	High
TC-03	Food placement, detection and snake growth	FR-07, FR-08, FR-09	Functional	High
TC-04	Score update and replacement food	FR-10, FR-11	Functional	High
TC-05	Collision detection and game over	FR-12, FR-13, FR-14, FR-15	Functional	High
TC-06	Restart after game over and state reset	FR-16, FR-17	Functional	High
TC-07	Safe window close and shutdown	FR-18, SR-05	Functional + Security	High
TC-08	Performance – frame rate	NFR-01	Non-functional (Performance)	Medium
TC-09	Usability – controls and restart are documented	NFR-02	Non-functional (Usability)	Medium
TC-10	Reliability – 10 consecutive start/restart cycles	NFR-03	Non-functional (Reliability)	High
TC-11	Maintainability – responsibilities in identifiable modules	NFR-04	Non-functional (Maintainability)	Medium

ID	Title	Requirements	Type	Priority
TC-12	Compatibility and build documentation	NFR-05, NFR-06	Non-functional (Compatibility / Documentation)	High
TC-13	Security – unsupported keys and direction validation	SR-01, SR-04, FR-06	Security	High
TC-14	Security – no command execution or unrelated file access	SR-02, SR-03	Security	High

11.2 Test Case Specifications

TC-01: Game start and initial display

Requirements	FR-01, FR-02, FR-03
Type / level	Functional — System / Integration
Priority	High
Preconditions	Game builds and launches; the start screen / initial state is shown. Configured initial snake position and length and the initial score are known (Section 6.3).
Test data	Initial snake position and length: [TBD – from config]. Initial score: [TBD – expected 0].
Test steps	1. Launch the game executable. 2. Perform the agreed start action ([TBD – e.g. Enter/Space]). 3. Confirm the game enters the playing state. 4. Check that the play-area boundary is visible. 5. Check that every initial snake segment is inside the boundary, at the configured start position and length. 6. Check that one food item is visible inside the boundary and does not overlap the snake. 7. Check that the current score is displayed at its initial value.
Expected result	The game enters the playing state; the boundary, snake, food and score are visible; every initial snake segment is inside the boundary.
Status / evidence	N (Not Run) | Tester: _____ Date: _____ Evidence: _____

TC-02: Keyboard direction input, movement and reversal rejection

Requirements	FR-04, FR-05, FR-06
Type / level	Functional — Unit + Integration + System
Priority	High
Preconditions	A round is in the playing state; the snake is moving in a known direction (e.g. right) and is at least 3 segments long.
Test data	Documented direction keys: [TBD – Arrow keys / WASD].
Test steps	1. While the snake moves right, press the 'up' key; confirm the snake turns up on the next move. 2. Press 'left', then 'down', then 'right'; confirm each turn happens as expected. 3. Observe at least 10 consecutive moves in one direction; confirm the head advances one cell per move and each body segment follows the one ahead. 4. While moving right, press 'left' (direct reverse); confirm the snake keeps moving right and does not reverse. 5. Repeat the reverse check for up→down, down→up and left→right. 6. Edge case: while moving right, press 'up' then 'left' in quick succession (within one move interval); confirm the snake never reverses into itself.
Expected result	A valid direction key changes direction per the control rules; the head advances and the body follows; every opposite-direction input is rejected, including two quick key presses within one move interval.
Status / evidence	N (Not Run) | Tester: _____ Date: _____ Evidence: _____

TC-03: Food placement, detection and snake growth

Requirements	FR-07, FR-08, FR-09
Type / level	Functional — Unit + Integration + System
Priority	High
Preconditions	A round is in the playing state; food is visible.
Test data	Growth per food: at least 1 segment. Use a seedable generator or debug hook to force food positions where needed.
Test steps	1. Note the snake length and food position at round start; confirm the food lies inside the play area and not on the snake. 2. Steer the snake so its head reaches the food. 3. Confirm exactly one food-consumption event is registered (not repeated on later moves). 4. Confirm the snake length increases by at least one segment. 5. Repeat at least five times. 6. Unit/debug check: force the snake to occupy most cells near the food and confirm food is never placed on a snake segment.
Expected result	Food always appears inside the play area on an unoccupied cell; one consumption event is registered per food item; the snake gains at least one segment each time.
Status / evidence	N (Not Run) | Tester: _____ Date: _____ Evidence: _____

TC-04: Score update and replacement food

Requirements	FR-10, FR-11
Type / level	Functional — Unit + Integration + System
Priority	High
Preconditions	A round is in the playing state; the configured food value is known.
Test data	Food value (points per food): [TBD – from config].
Test steps	1. Note the displayed score and the current food position. 2. Collect the food. 3. Confirm the displayed score increases by exactly the configured food value. 4. Confirm a new food item appears at a valid, unoccupied position inside the play area. 5. Repeat for at least five consecutive foods; confirm the score total equals food count × food value and exactly one food item is on screen each time.
Expected result	The displayed score increases by the configured food value after each pickup, and a new food item appears at a valid unoccupied position every time.
Status / evidence	N (Not Run) | Tester: _____ Date: _____ Evidence: _____

TC-05: Collision detection and game over

Requirements	FR-12, FR-13, FR-14, FR-15
Type / level	Functional — Unit + Integration + System
Priority	High
Preconditions	A round is in the playing state on open ground. A debug start layout and a way to place or disable food are available so the snake can be set to an exact length and shape. Food is kept off the test route (and off the cells beside it) so eating cannot change the fixed snake length. Per the SAD, a head moving into the cell the tail is leaving in the same move is permitted (not a collision).
Test data	Boundary: top, bottom, left and right edges. Self-collision layout A: total length 6 (head + 5 body), moving right. Departing-tail layout B: total length exactly 4 (head + 3 body), moving right. Example grid coordinates (x, y), y increasing downward.
Test steps	1. Steer the snake into the top edge; record what happens. 2. Start a new round and repeat for the bottom, left and right edges.

	3. Before steps 4–6, place the food well away from the planned route (for example in a far corner) or disable it, and confirm the snake length is unchanged after each run; discard any run in which food was eaten. 4. Self-collision with a non-tail segment (layout A): head at (5,5), body (4,5), (3,5), (2,5), (1,5), (0,5), moving right. Press up, then left, then down in successive moves. The head enters (4,5), which is occupied by a body segment that is not the tail. Confirm the game-over state begins. 5. Departing-tail case (layout B): head at (5,5), body (4,5), (3,5), (2,5), moving right. Press up, then left, then down in successive moves. On the last move the head enters (4,5), the cell the tail vacates in the same move. Confirm the game does NOT end and the snake keeps moving (SAD rule). 6. Repeat the layout A sequence with a longer snake (length 8 or more) and confirm self-collision is still detected. 7. For every fatal collision above, confirm the snake stops moving immediately. 8. Confirm the game-over message and the final score are visible, and that the final score equals the score at the moment of collision.
Expected result	Boundary collisions and self-collision with any non-tail body segment trigger the game-over state; the snake stops; the game-over message and correct final score are visible. Entering the departing tail's cell does not end the game.
Status / evidence	N (Not Run) | Tester: _____ Date: _____ Evidence: _____

TC-06: Restart after game over and state reset

Requirements	FR-16, FR-17
Type / level	Functional — System
Priority	High
Preconditions	The game is in the game-over state after a fatal collision, with a non-zero score and a snake that is longer than its initial length.
Test data	Restart action: [TBD – e.g. Enter/R]. Initial state values from config.
Test steps	1. From the game-over screen, perform the restart action. 2. Confirm a new round starts without relaunching the application. 3. Confirm the snake is back at the defined initial position and length. 4. Confirm the direction is reset to the initial direction. 5. Confirm the score is reset to its initial value. 6. Confirm new food is placed at a valid position.
Expected result	A new round begins from the defined initial state and score, without relaunching the application.
Status / evidence	N (Not Run) | Tester: _____ Date: _____ Evidence: _____

TC-07: Safe window close and shutdown

Requirements	FR-18, SR-05
Type / level	Functional + Security — System
Priority	High
Preconditions	Game launched on the test computer; a sanitizer-enabled build (AddressSanitizer/UBSan) is available.
Test data	Window close button (and the platform close shortcut, e.g. Alt+F4).
Test steps	1. During active gameplay, close the window with the close button; confirm the application exits without a crash or error dialog. 2. Relaunch, reach the game-over state and close the window; confirm a clean exit. 3. Relaunch and close the window while a direction key is held down; confirm a clean exit. 4. Close the window immediately after launch, before starting a round; confirm a clean exit. 5. Repeat steps 1–2 with the sanitizer-enabled build; confirm no sanitizer reports and a normal exit code.
Expected result	Every close scenario exits the application without an unhandled crash, hang, error dialog or sanitizer report.
Status / evidence	N (Not Run) | Tester: _____ Date: _____ Evidence: _____

TC-08: Performance – frame rate

Requirements	NFR-01
Type / level	Non-functional (Performance) — System
Priority	Medium
Preconditions	Release (or agreed) build running on the agreed test computer with no heavy background load.
Test data	Test computer specification: [TBD – record CPU, RAM, GPU, OS]. Measurement: frame counter or on-screen FPS overlay / log output.
Test steps	1. Record the test computer specification in the test log. 2. Start a round and play normally (turning and collecting food) for 60 seconds while counting rendered frames. 3. Compute average FPS = total frames ÷ 60 s. 4. Repeat the run three times and record each average.
Expected result	The measured average is at least 30 FPS during the defined 60-second run (NFR-01) in every run.
Status / evidence	N (Not Run) | Tester: _____ Date: _____ Evidence: _____

TC-09: Usability – controls and restart are documented

Requirements	NFR-02
Type / level	Non-functional (Usability) — Acceptance
Priority	Medium
Preconditions	README and in-game instructions are available. The reviewer has not worked on the code.
Test data	Reviewer name and date recorded in the test log.
Test steps	1. Give the reviewer only the README and the game (no verbal help). 2. Ask them to state how to start a round, steer the snake and restart after game over. 3. Ask them to play one round; note any confusion or errors. 4. Record whether the controls and restart action are described in the game interface or the README.
Expected result	The reviewer correctly identifies the controls and the restart procedure from the supplied instructions alone.
Status / evidence	N (Not Run) | Tester: _____ Date: _____ Evidence: _____

TC-10: Reliability – 10 consecutive start/restart cycles

Requirements	NFR-03
Type / level	Non-functional (Reliability) — System
Priority	High
Preconditions	Game running on the test computer.
Test data	One cycle = start a round, play to a fatal collision, restart.
Test steps	1. Start the first round and play until a fatal collision. 2. Restart and repeat until 10 consecutive start/game-over/restart cycles are completed without relaunching. 3. After each cycle confirm the game is responsive and the state (score, snake, food) was reset. 4. Record any crash, freeze or visible glitch with its cycle number. Optionally watch memory use in the OS task monitor for steady growth.
Expected result	All 10 cycles complete without a crash.
Status / evidence	N (Not Run) | Tester: _____ Date: _____ Evidence: _____

TC-11: Maintainability – responsibilities in identifiable modules

Requirements	NFR-04

Type / level	Non-functional (Maintainability) — Code review
Priority	Medium
Preconditions	Source code is committed to the repository.
Test data	Review checklist: movement, food, collision, game control, rendering.
Test steps	1. Open the source tree and list the classes/modules. 2. For each responsibility (movement, food, collision, game control, rendering) locate the class or module that implements it. 3. Check that each module has a clear interface (header / public methods) and that rendering is not mixed into collision or movement logic where practical. 4. Record findings and any deviations, with reasons, in the review checklist.
Expected result	Code review can identify the movement, food, collision, game-control and rendering responsibilities in identifiable modules or classes.
Status / evidence	N (Not Run) | Tester: _____ Date: _____ Evidence: _____

TC-12: Compatibility and build documentation

Requirements	NFR-05, NFR-06
Type / level	Non-functional (Compatibility / Documentation) — System + Review
Priority	High
Preconditions	Source and README are committed. A teammate who did not write the README is available.
Test data	Documented compiler, SFML version and build commands from the README.
Test steps	1. Check that the README lists the required tools, the SFML dependency and the build/run steps. 2. Clone the repository fresh into an empty folder on a machine matching the documented environment. 3. A teammate follows only the README to prepare the environment, build and launch the game. 4. Confirm a clean build succeeds and the game launches. 5. Record any missing or incorrect README step as a defect.
Expected result	A clean build and launch succeed in the documented environment, and a teammate can follow the README to prepare the environment and build and launch the game.
Status / evidence	N (Not Run) | Tester: _____ Date: _____ Evidence: _____

TC-13: Security – unsupported keys and direction validation

Requirements	SR-01, SR-04, FR-06
Type / level	Security — Unit + System
Priority	High
Preconditions	A round is in the playing state. Sanitizer-enabled build available.
Test data	Unsupported keys: any key not in the documented control set (letters, function keys, Tab, etc.).
Test steps	1. During play, press each unsupported key one by one; confirm it is ignored and gameplay stays responsive. 2. Mash random keys for about 10 seconds; confirm no crash or freeze. 3. Press the start/restart key during play and a direction key on the game-over screen; confirm these invalid state changes are ignored or handled safely. 4. Repeat the direct-reversal checks from TC-02 (all four opposite pairs); confirm every reverse is rejected before it changes the game state. 5. Press two quick keys within one move interval that together would reverse the snake; confirm no self-reversal. 6. Run steps 1–5 with the sanitizer-enabled build; confirm no errors.
Expected result	Unsupported keys are ignored and gameplay remains responsive; invalid state changes do not crash the application; immediate reverse-direction input is always rejected.
Status / evidence	N (Not Run) | Tester: _____ Date: _____ Evidence: _____

TC-14: Security – no command execution or unrelated file access

Requirements	SR-02, SR-03
Type / level	Security — Code review + System
Priority	High
Preconditions	Source code available; tools available: grep or cppcheck, and strace (Linux) or Process Monitor (Windows).
Test data	Search terms: system, popen, exec*, fork, ShellExecute, CreateProcess; file calls: fopen, ifstream, ofstream, filesystem.
Test steps	1. Search the code base for operating-system command calls; for each hit, trace whether any player-input path can reach it. Record result. 2. List every file open/read/write call; confirm each targets only the game's own assets or configuration. 3. Run the game under strace -f -e trace=file (or Process Monitor) through a full round including restart and close. 4. Review the trace; confirm no read or write of unrelated user files (only game assets and required system/SFML libraries). 5. Record the tool output as evidence.
Expected result	Code review finds no player-input path that executes operating-system commands and no unrelated user-file access during normal gameplay.
Status / evidence	N (Not Run) | Tester: _____ Date: _____ Evidence: _____